

Linguagem de Programação II

Modelagem e Programação Orientada a Objetos (POO)

Prof. Eduardo Paiva

CESFI / UDESC - Engenharia de Petróleo

Semestre 2026/1

Objetivos da Aula

- Compreender o paradigma da Programação Orientada a Objetos (POO).
- Diferenciar **Classes** (abstrações) de **Objetos** (instâncias).
- Desmistificar a sintaxe do Python: `class`, `__init__` e `self`.
- Integrar a leitura de dados (CSV/LAS via Pandas) com a criação automática de Objetos.

O Paradigma da POO: Uma Nova Forma de Pensar

Na Engenharia, nós lidamos com sistemas interconectados.

- **A abordagem Procedural (Tradicional):** Foco nas ações. Criamos listas de variáveis soltas (ex: uma lista de densidades, outra de profundidades) e passamos por funções.
- **A abordagem Orientada a Objetos:** Foco nas **entidades**. Nós agrupamos os dados (Atributos) e as ações (Métodos) em um único "pacote" lógico.

Em vez de termos dezenas de variáveis soltas no código, teremos um Objeto Poço que sabe sua própria profundidade e calcula sua própria pressão.

A Analogia Definitiva: A Forma de Bolo

Para entender POO, precisamos separar o "Projeto" do "Produto Físico":

A Classe (A Forma de Bolo)

É apenas o molde. A palavra reservada `class` no Python avisa o computador: "*Estou desenhando a planta baixa de um novo tipo de dado*". Ela não tem valores reais ainda, apenas diz quais características existirão.

O Objeto (O Bolo Pronto)

É a materialização da classe (a **instância**). Usando uma mesma forma, podemos fazer vários bolos diferentes (de chocolate, de cenoura). Na engenharia, usando a mesma Classe Fluido, criamos Objetos reais: *Lama Baritada*, *Emulsão Invertida*, etc.

A Sintaxe Mágica: Entendendo o `__init__` e o `self`

Para criar nosso Objeto, o Python precisa de um construtor. Pense no `__init__` como a **Ficha de Matrícula** do seu objeto.

```
class Fluido:
    # O __init__ roda automaticamente quando o objeto nasce
    def __init__(self, nome_digitado, densidade_digitada):
        self.nome = nome_digitado
        self.densidade = densidade_digitada
```

- `def __init__`: É o método (função) inicializador.
- `self`: Significa "este objeto específico". Ao dizer `self.nome = nome_digitado`, o Python pega o que você digitou e cola uma "etiqueta" permanente no objeto dizendo: "O meu nome é este".

A Área de Testes: Tirando do Papel

Agora vamos instanciar (criar) os objetos reais e dar ordens a eles usando o **ponto (.)**.

```
# 1. Chamando a "fabrica" (A Classe) para criar o Objeto
lama_pesada = Fluido(nome_digitado="Lama Baritada", densidade_digitada=1800)
lama_leve = Fluido(nome_digitado="Lama Base Agua", densidade_digitada=1000)

# 2. Acessando os atributos do objeto com o ponto
print(lama_pesada.nome) # Imprime: Lama Baritada
print(lama_leve.densidade) # Imprime: 1000
```

Nota: Diferente do C++, no Python nativo não precisamos importar bibliotecas prévias (como NumPy ou Pandas) para que as classes e objetos funcionem.

Criando Métodos (Ações do Objeto)

Além de guardar dados (Atributos), os objetos sabem fazer coisas (Métodos).

```
class Fluido:
    def __init__(self, nome, densidade, comportamento):
        self.nome = nome
        self.densidade = densidade
        self.comportamento = comportamento

    # Este e um Metodo! Uma acao exclusiva da classe Fluido.
    def avaliar_reologia(self):
        if self.comportamento == "Comportamento Dilatante":
            return "Atencao: Viscosidade aparente subindo com o cisalhamento!"
        return "Comportamento estavel."
```

O Poder da Integração: Pandas + POO

Na vida real, não digitamos fluido por fluido. Usamos o Pandas para ler dezenas de linhas de um arquivo (CSV ou LAS) e transformar cada linha em um Objeto automaticamente!

```
import pandas as pd

# 1. Lemos a planilha do laboratorio
df = pd.read_csv('banco_fluidos.csv')
lista_de_fluidos = []

# 2. O laço 'for' passa linha por linha criando objetos em lote
for indice, linha in df.iterrows():
    novo_objeto = Fluido(
        nome = linha['Nome'],
        densidade = linha['Densidade'],
        comportamento = linha['Tipo']
    )
    lista_de_fluidos.append(novo_objeto) # Guarda o objeto na lista
```

A Missão da Semana:

- 1 Modelem conceitualmente um sistema Poço-Fluido.
- 2 Utilizem o arquivo `dados_pocos.csv` disponibilizado no Moodle.
- 3 Criem um script que leia o arquivo via Pandas, limpe os dados e instancie os Objetos Poço e Fluido automaticamente.
- 4 Calculem a pressão de fundo de cada poço usando os métodos construídos.

Critério PAP (Performance e Atitude Profissional): Códigos limpos, nomes de variáveis que façam sentido e objetos bem encapsulados garantem a pontuação máxima.